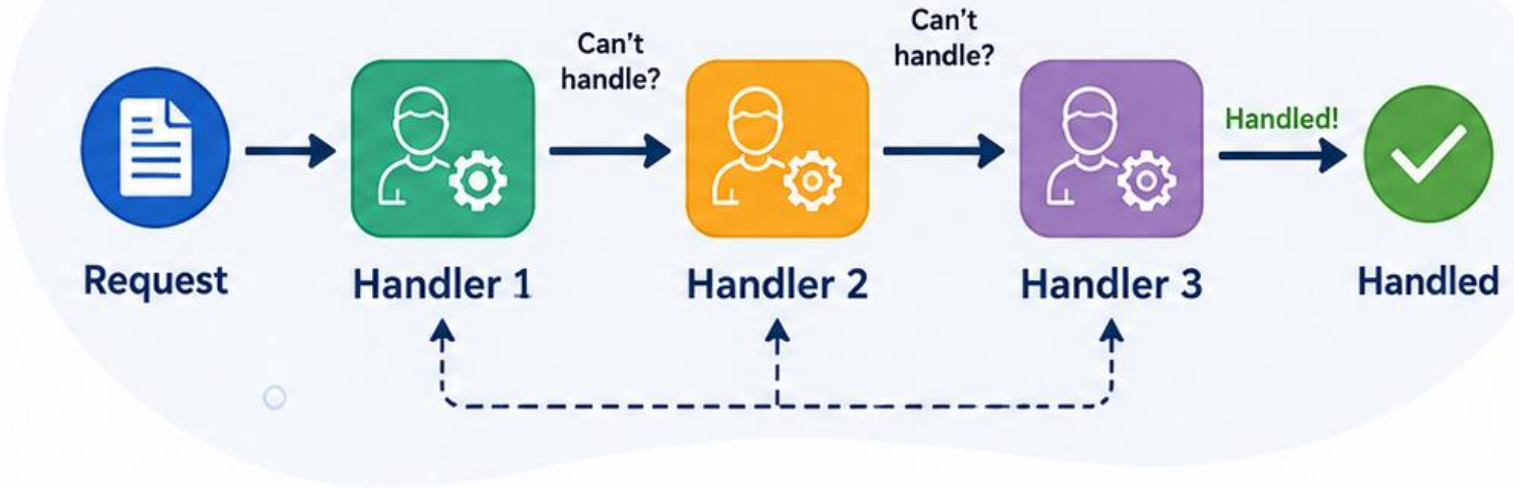
Chain of Responsibility

Design Pattern

A Behavioral Design Pattern in OOP

Key Idea:

A request is passed through a chain of potential handlers until one of them handles it.



Decouples the sender of a request from its receiver by giving more than one object a chance to handle the request.

Where Are We?

Design Pattern Categories



Creational Patterns

- [x] Factory Method
- [ ] Abstract Factory
- [x] Singleton
- [ ] Prototype
- [ ] Builder

Structural Patterns

- [x] Adapter
- [ ] Bridge
- [ ] Composite
- [x] Decorator
- [x] Facade
- [ ] Flyweight
- [x] Proxy

Behavioral Patterns

- [x] Chain of Responsibility ← Today
- [ ] Command
- [ ] Interpreter
- [ ] Iterator
- [ ] Mediator
- [ ] Memento
- [ ] Observer
- [ ] State
- [ ] Strategy
- [x] Template
- [ ] Visitor

Covered · In Progress (Today) · Upcoming

★ We have covered: Singleton, Factory Method, Adapter, Decorator, Facade, Proxy
Today, we cover the first Behavioral pattern – **Chain of Responsibility.** Template (Behavioral) was also covered.

Our Journey So Far

Patterns We Have Discussed

Pattern	Category	Main Purpose
Singleton	Creational	Ensure a single instance
Factory Method	Creational	Delegate object creation
Facade	Structural	Simplify access to a subsystem
Adapter	Structural	Make incompatible interfaces work together
Proxy	Structural	Control access to another object
Decorator	Structural	Add responsibilities dynamically
Template Method	Behavioral	Define the skeleton of an algorithm, defer some steps to subclasses



Quick Revisit Activity

Think – Pair – Share

For each scenario, identify the most suitable pattern.

Scenario A	Only one database connection manager should exist.	Answer: **Singleton Pattern**
Scenario B	A legacy payment system must work with a new payment interface.	Answer: **Adapter Pattern**
Scenario C	Additional features must be added dynamically to a notification service.	Answer: **Decorator Pattern**
Scenario D	A client wants simple access to several complex subsystems.	Answer: **Facade Pattern**
Scenario E	A request may be handled by one of several possible objects.	Answer: **None directly.** (See Reflection →)



Reflection

Which of the previously discussed patterns best solves Scenario E?

Answer:
None directly.

Moving to Behavioral Patterns

Until Now...

Creational Patterns

How should objects be created?



Structural Patterns

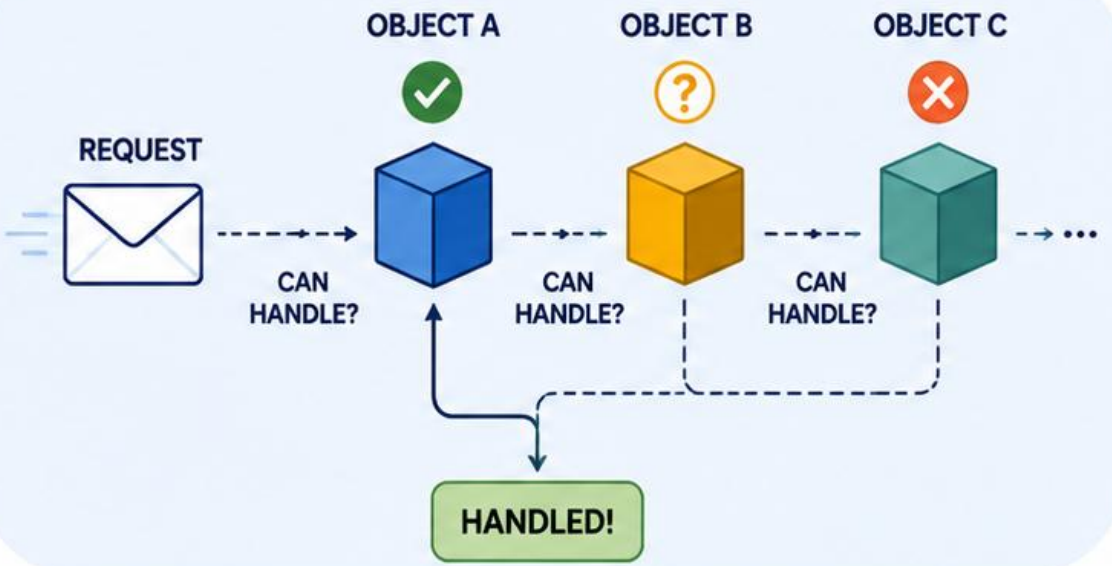
How should objects be connected or organised?



Behavioral Patterns

How should objects communicate and distribute responsibilities?





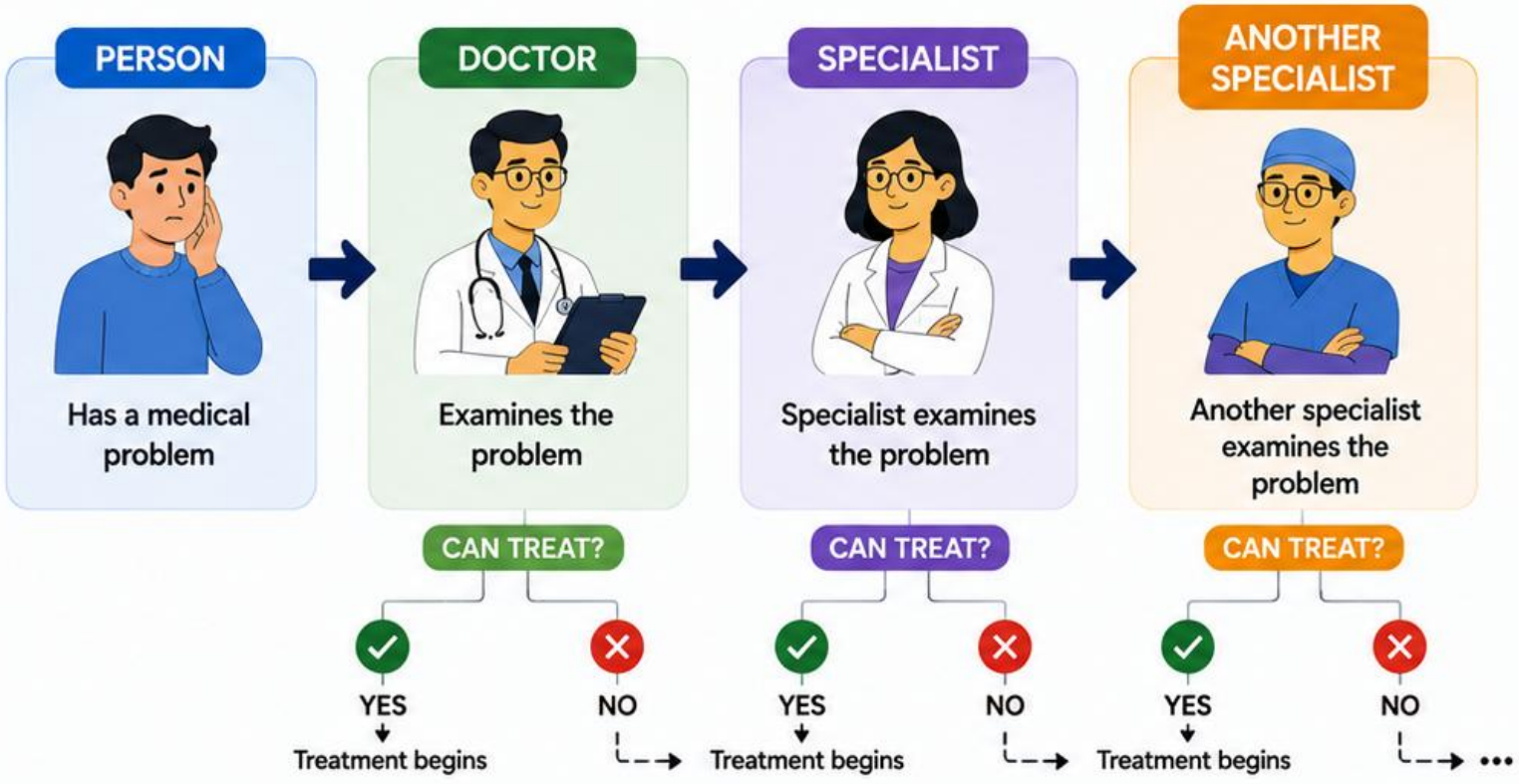
Today's Question

"When multiple objects may be able to process a request, who should decide which object handles it?"

Chain of Responsibility

Use your existing visual:

Person → Doctor → Specialist → Another Specialist



The request continues along the chain until someone appropriate handles it.

Discussion

1. You visit a doctor.
2. The doctor examines the problem.
3. If the doctor can treat it, treatment begins.
4. Otherwise, you may be referred to a specialist.
5. The request continues until someone **appropriate handles it.**

Key Idea

The patient does not need to know in advance exactly which specialist will handle the problem.

What Is the Chain of Responsibility Pattern?

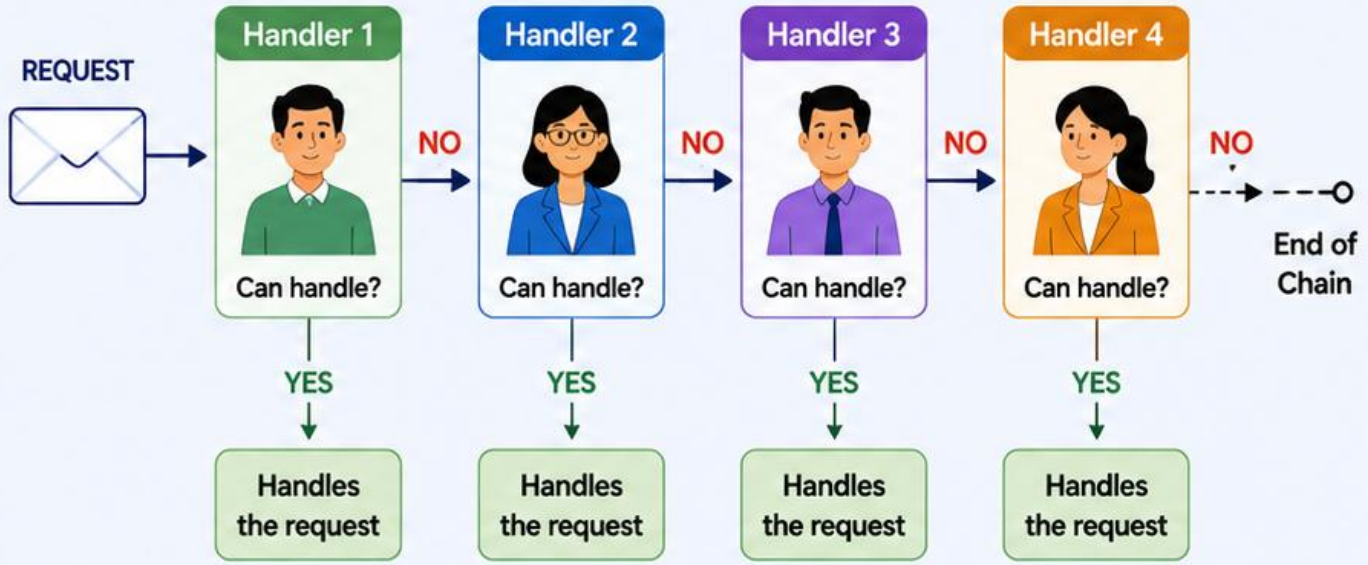
Chain of Responsibility Pattern

A Behavioral Design Pattern

- Avoids coupling the sender of a request to its receiver.
- More than one object may be able to handle a request.
- The receiving objects are organised into a chain.
- Each object decides whether it can handle the request.
- If it cannot, the request is passed to the next object.

The chain ends when:

- A handler processes the request, **or**
- The end of the chain is reached.



The request travels along the chain until some handler is able to process it.

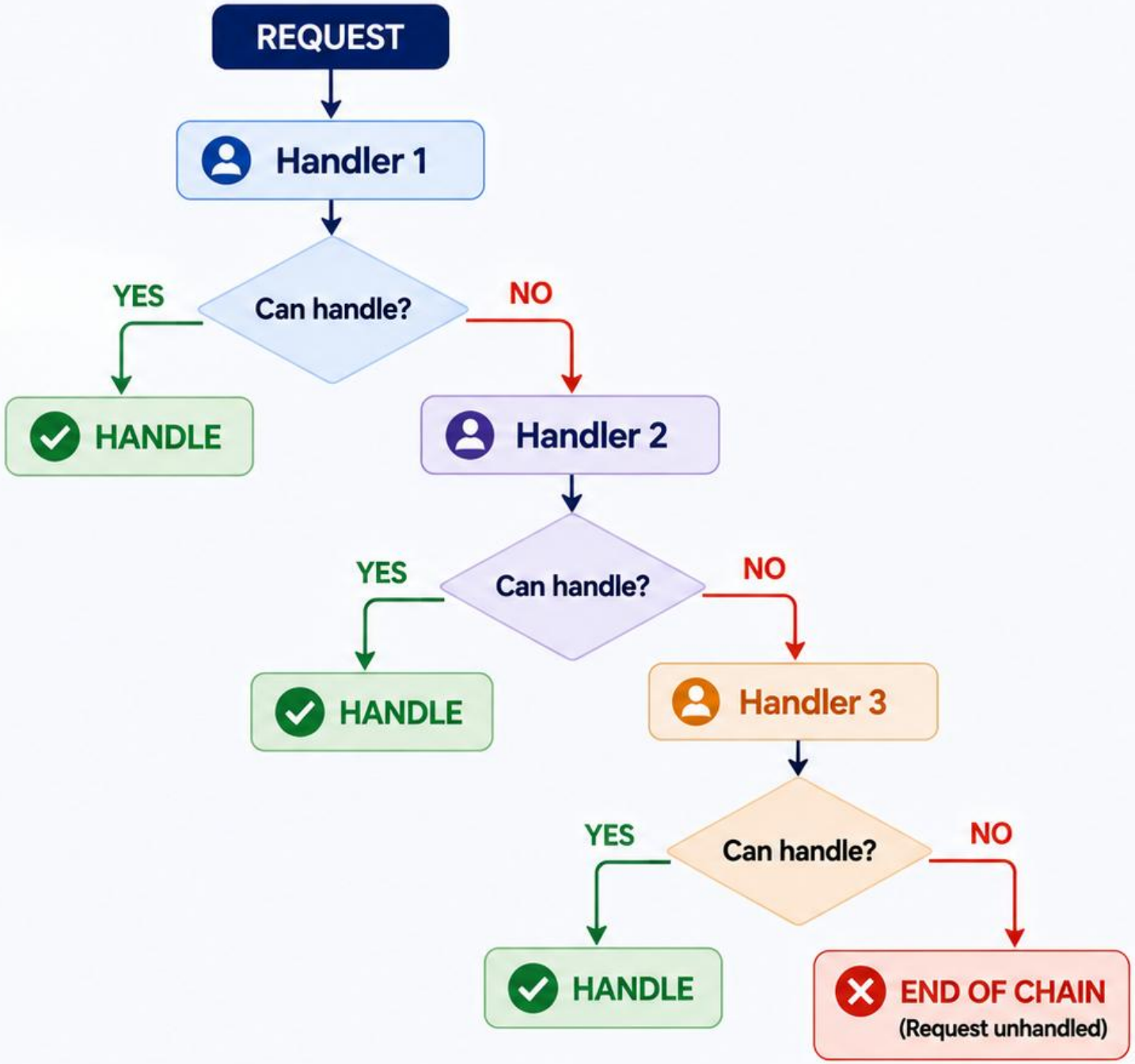
The Core Idea

Every Handler Has Two Choices

Handle the request

OR

Pass the request to the next handler



OOP Essentials for Chain of Responsibility

Core Object-Oriented Principles You Need

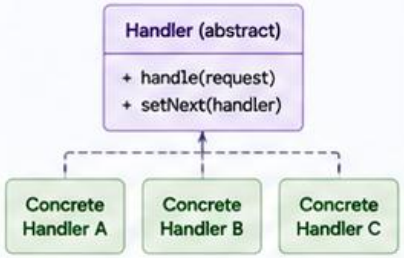
Chain of Responsibility relies on fundamental OOP concepts. Let's revisit the key principles that enable this pattern.

Why OOP Matters Here

- Promotes modular, reusable code
- Encourages loose coupling
- Makes systems flexible and extensible
- Easier to maintain and test

1 Abstraction

Focus on what an object does, not how it does it. Define a common interface (or abstract class) that declares the operations.



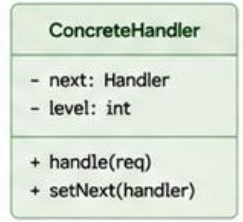

```
abstract class Handler {
   protected Handler next;

   public void setNext(Handler next) {
      this.next = next;
   }

public abstract void handle(Request req);
}
```

2 Encapsulation

Hide internal details and expose only what is necessary. Handlers manage their own logic and next reference safely.

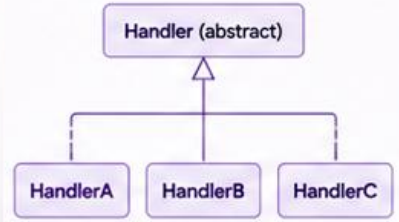



```
class ConcreteHandler extends Handler {
  private int level; // encapsulated

  public ConcreteHandler(int level) {
    this.level = level;
  }
  public void handle(Request req) {
    // handle or pass to next
  }
}
```

3 Inheritance

Concrete handlers inherit from a common base (abstract class or interface) and reuse its structure.



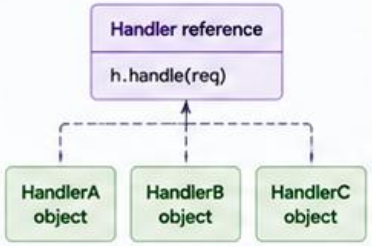

```
class HandlerA extends Handler {
  public void handle(Request req) {
    // specific handling
  }
}

class HandlerB extends Handler { ... }

class HandlerC extends Handler { ... }
```

4 Polymorphism

Treat different handler objects uniformly through the base type. The correct method is invoked at runtime (dynamically).




```
Handler h = new HandlerA();
h.handle(req);   // calls HandlerA.handle()

h = new HandlerB();
h.handle(req);   // calls HandlerB.handle()

h = new HandlerC();
h.handle(req);   // calls HandlerC.handle()
```

5 Association (Has-A Relationship)

Handlers are linked together. Each handler "has" a reference to the next handler in the chain.



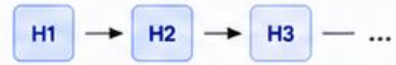

```
public void setNext(Handler next) {
    this.next = next;
}

if (next != null) {
     next.handle(req); // pass to next handler
}
```

6 Composition & Aggregation

The client composes the chain by linking handlers. Handlers aggregate the next handler reference.






```
Handler h1 = new HandlerA();
Handler h2 = new HandlerB();
Handler h3 = new HandlerC();
}

h1.setNext(h2);
h2.setNext(h3);
```

Key Takeaway

These OOP principles work together to make Chain of Responsibility:

- **Modular** — Easy to extend
- **Loose Coupling** — Less dependency
- **Flexible** — Change at runtime
- **Maintainable** — Clean structure

Problem 1: IT Support Escalation

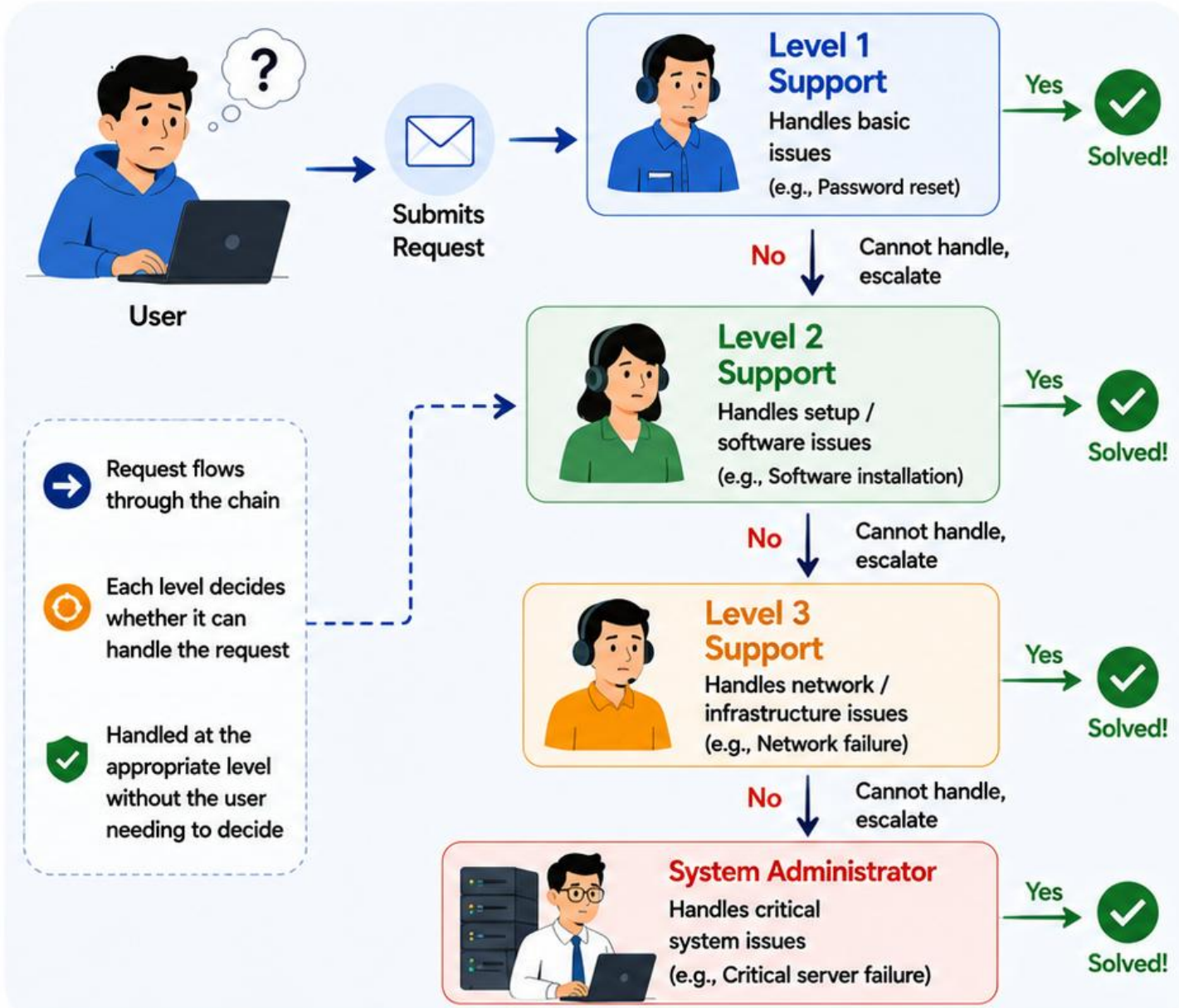
Scenario

A university IT support system receives different requests.

Request	Support Level
Password reset	Level 1
Software installation	Level 2
Network failure	Level 3
Critical server failure	System Administrator

Question

Should the user decide exactly which support team should receive the request?



Traditional Approach

The client may contain logic such as:

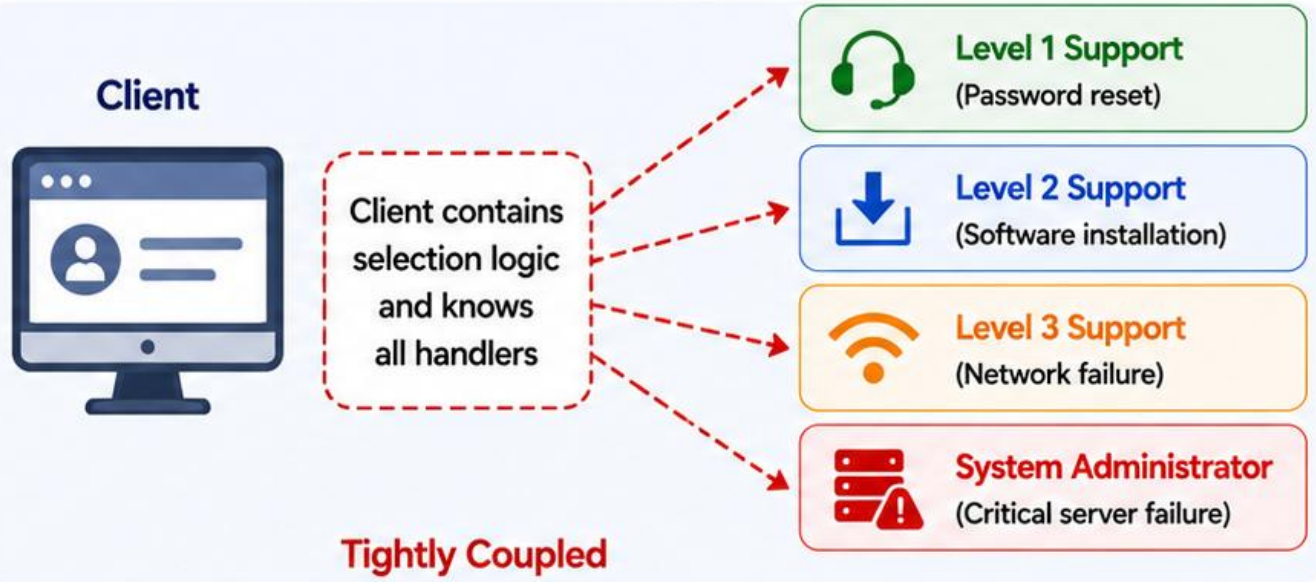
```
if (requestType.equals("PASSWORD")) {
    level1.handle(request);
}

else if (requestType.equals("SOFTWARE")) {
    level2.handle(request);
}

else if (requestType.equals("NETWORK")) {
    level3.handle(request);
}

else {
    administrator.handle(request);
}
```

The client must know exactly who can handle each request and make the decision itself.



Problems

- Client knows every handler.
- Client is tightly coupled to the support system.
- Adding a new support level may require modifying the client.
- Responsibility selection logic becomes centralised.

Using a Chain

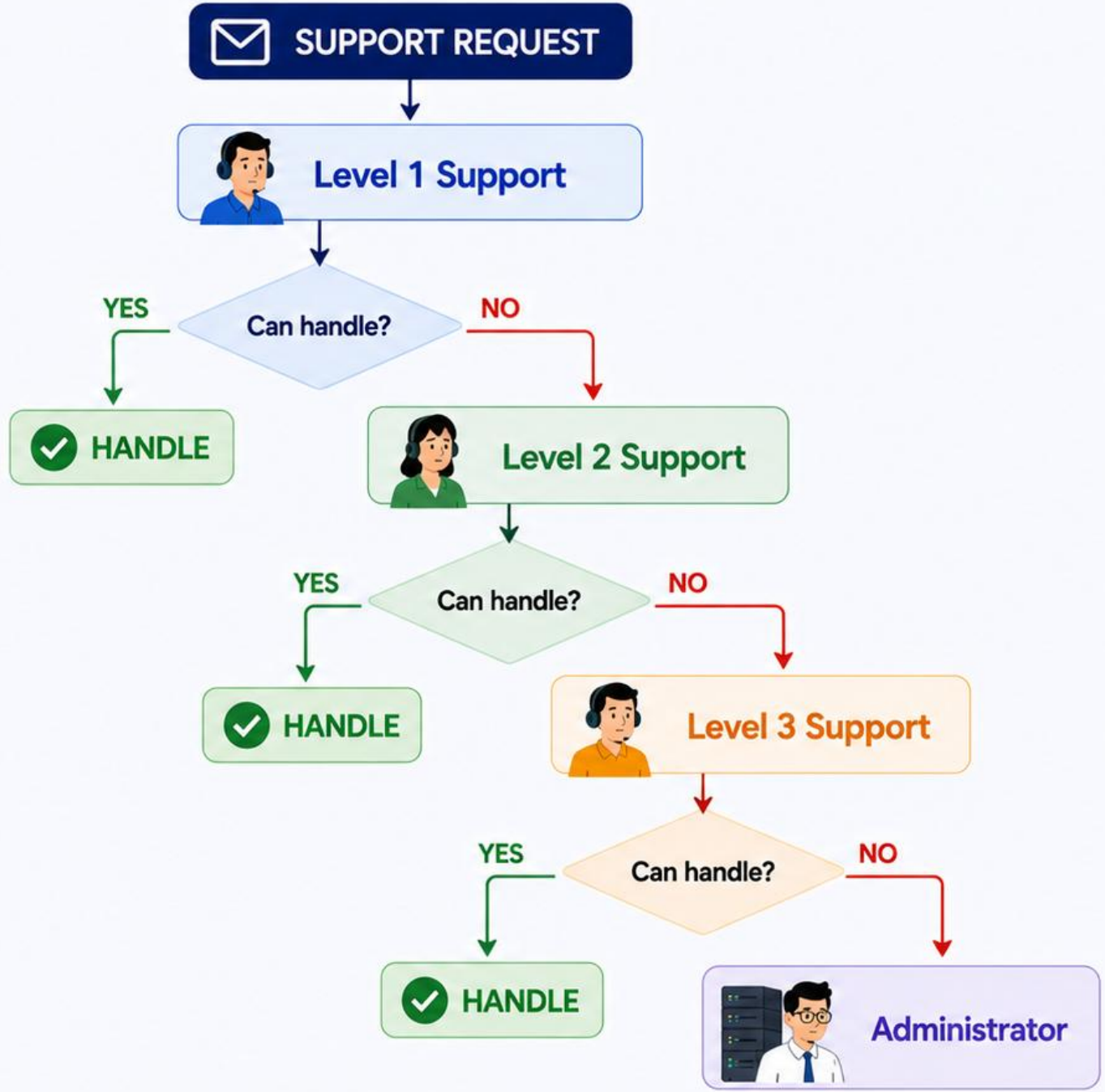
The Client Only Knows

Where to **send the request**.

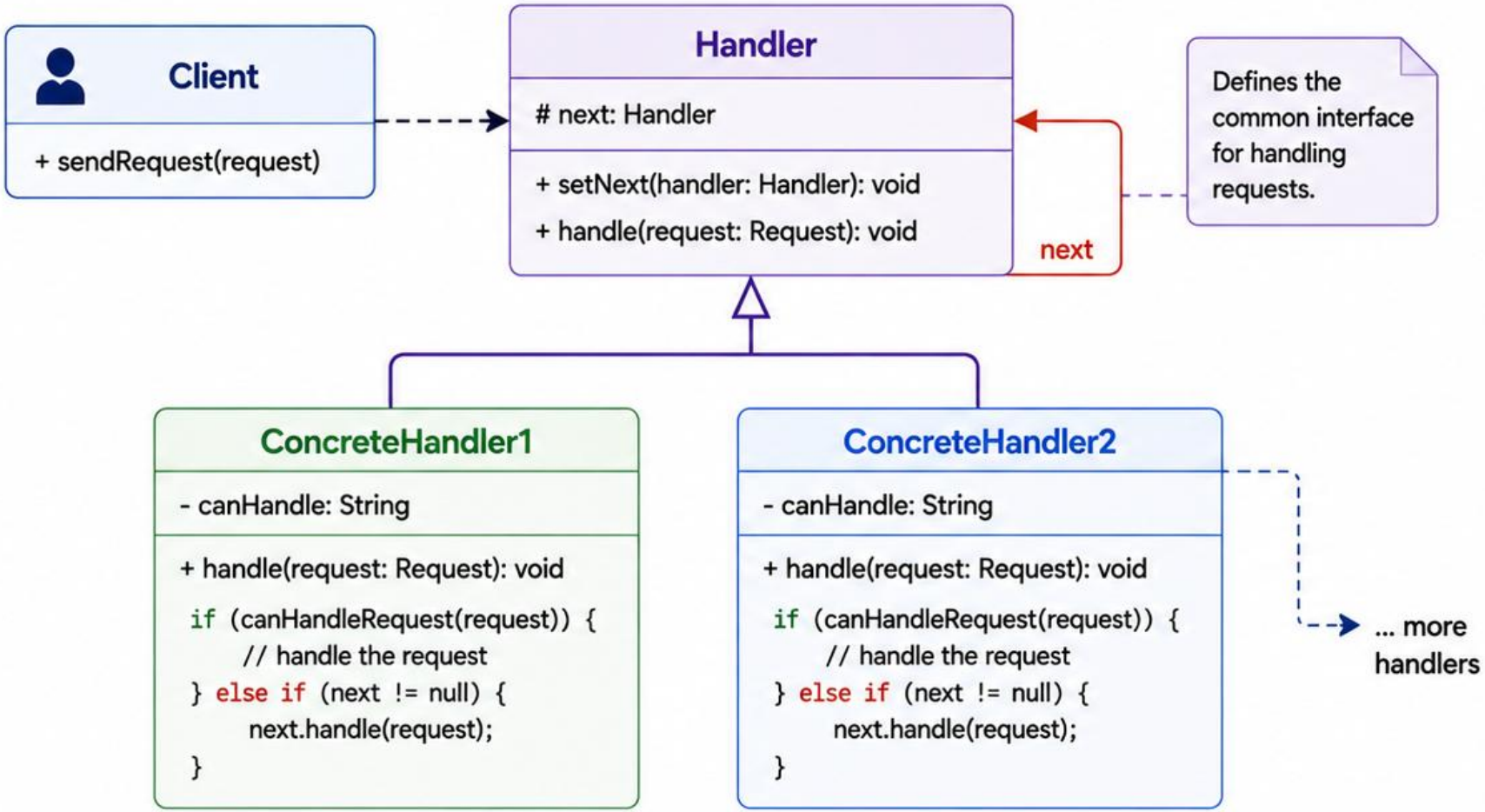
The client **does not** need to know:

Who will **ultimately handle** it.

The request is **passed along** the chain until someone can handle it.



UML Structure



Handlers are **linked as a chain.** Each handler decides either to handle the request or to pass it to the next handler in the chain.

Main Participants

Client
Sends the request to the first handler.

Handler
Defines the common method for handling requests.

ConcreteHandler1
Handles requests that fall within its responsibility.

ConcreteHandler2
Handles another category of requests.

Next Handler
If the current handler cannot process the request:

Forward it to the next handler.

Java Implementation: Request

SupportRequest

```java
public class SupportRequest {
    private int level;
    private String description;

    public SupportRequest(
        int level,
        String description) {
        this.level = level;
        this.description = description;
    }

    public int getLevel() {
        return level;
    }

    public String getDescription() {
        return description;
    }
}
```

Purpose

Encapsulates all information about a support request so that handlers can decide whether they can process it.

Key Data

- level – indicates the type/priority of the request
- description – explains the problem in detail

Reflection

What information should a handler receive in order to decide whether it can process the request?

A handler needs:

Level / Priority
To know the category or severity of the request.

Description
To understand the problem in detail.

Additional Context (optional)
e.g., timestamp, user, system affected, attachments, etc.

With this information, the handler can decide: HANDLE or PASS to the next.

Java Implementation: Abstract Handler

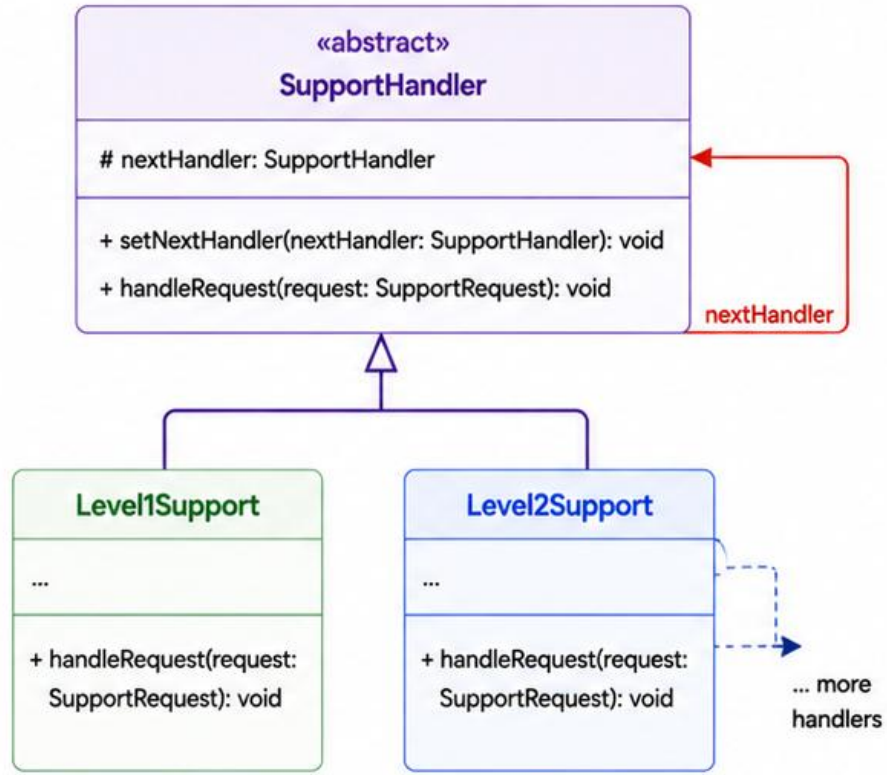
SupportHandler (Abstract Class)

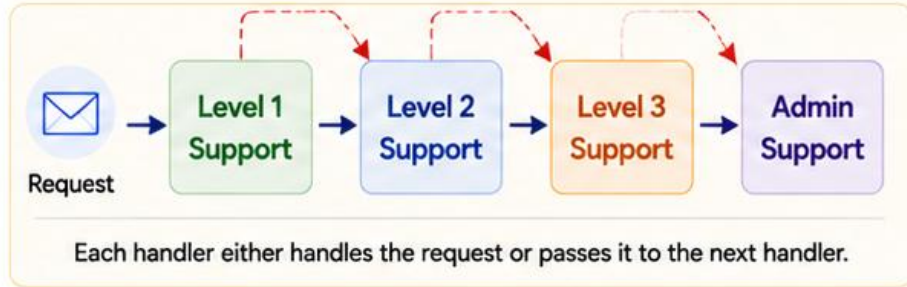
```java
public abstract class SupportHandler {
  // Reference to the next handler in the chain
  protected SupportHandler nextHandler;

  // Set the next handler in the chain
  public void setNextHandler(
       SupportHandler nextHandler) {
       this.nextHandler = nextHandler;
  }

  // Handle the request
  public abstract void handleRequest(
       SupportRequest request);
}
```

This class defines the structure of the chain and the common operation for all handlers.





Discussion

Why is this used?

```
SupportHandler nextHandler;
```

instead of:

```
Level2Support nextHandler;
```

Think About

Abstraction
SupportHandler is the abstraction. It does not depend on any specific concrete class.

Polymorphism
Any concrete handler (Level1Support, Level2Support, etc.) can be assigned to nextHandler and treated uniformly.

Flexibility
New handler types can be added to the chain without changing the abstract handler or the client.

This makes the chain **extensible, maintainable, and reusable.**

Java Implementation: Level 1 Support

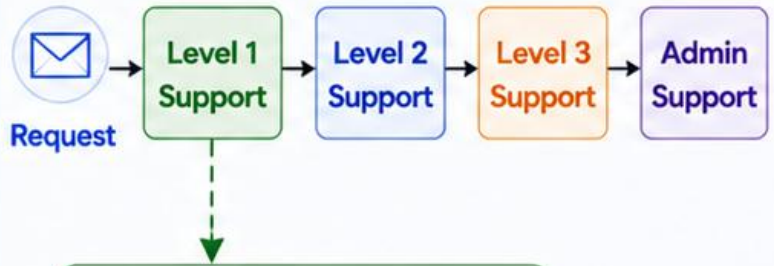
Level1Support.java

```java
public class Level1Support
    extends SupportHandler {

 @Override
 public void handleRequest(
    SupportRequest request) {
    if (request.getLevel() == 1) {
        System.out.println(
            "Level 1 handled: " +
            request.getDescription());
    } else if (nextHandler != null) {
        nextHandler.handleRequest(request);
    }
 }
}
```

Level1Support can handle requests whose **level is 1**.
Otherwise, it **forwards** the request to the next handler in the chain.

Where Level 1 Fits in the Chain



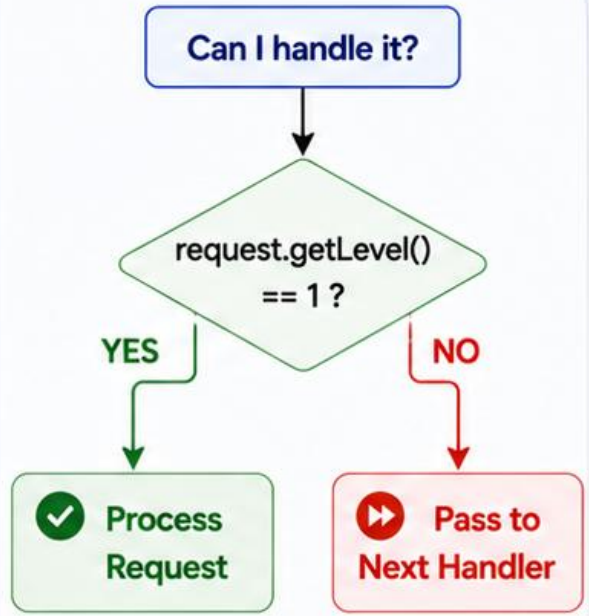
Level 1 Responsibility

Handles basic issues such as password reset, account lock, access problems, etc.

Example

- Password reset
- Account unlock
- Basic user access issues

Key Logic



YES → Process request

NO → Pass to next handler

Java Implementation: Other Handlers

Level2Support.java

```java
public class Level2Support
    extends SupportHandler {

 @Override
 public void handleRequest(
     SupportRequest request) {
     if (request.getLevel() == 2) {
         System.out.println(
             "Level 2 handled: " +
             request.getDescription());
     } else if (nextHandler != null) {
         nextHandler.handleRequest(request);
     }
 }
}
```

What this does

Handles requests with level == 2.
Otherwise, passes the request to the next handler.

Level3Support.java

```java
public class Level3Support
    extends SupportHandler {

 @Override
 public void handleRequest(
     SupportRequest request) {
     if (request.getLevel() == 3) {
         System.out.println(
             "Level 3 handled: " +
             request.getDescription());
     } else if (nextHandler != null) {
         nextHandler.handleRequest(request);
     }
 }
```

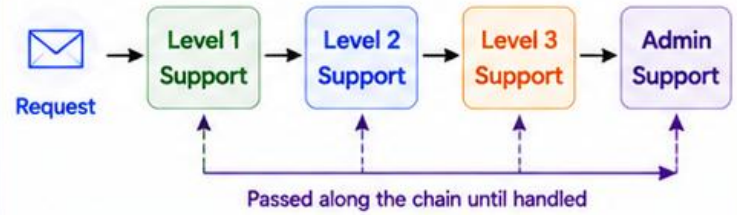
Students: Identify the changes

Compare Level2Support with Level3Support.
What is different? What is the same?

	Level 2	Level 3
Condition	`request.getLevel() == 2`	`request.getLevel() == 3`
Message	`"Level 2 handled: ..."`	`"Level 3 handled: ..."`
Other logic	Same	Same

Pattern in all concrete handlers

- Check: Can I handle this request?
- If YES: Process the request.
- If NO: Pass to the next handler.



Reflection Question

Do we need to change the SupportHandler class when adding a new support level?

Think about: abstraction, extensibility, Open/Closed Principle.

Key Takeaway: Each handler handles its own responsibility and passes along the request if it cannot handle it.
New handlers can be added **without modifying** existing handlers or the `SupportHandler` class.

Building the Chain

We create the handlers and link them in order using **setNextHandler()**.
Each handler knows only its next handler.

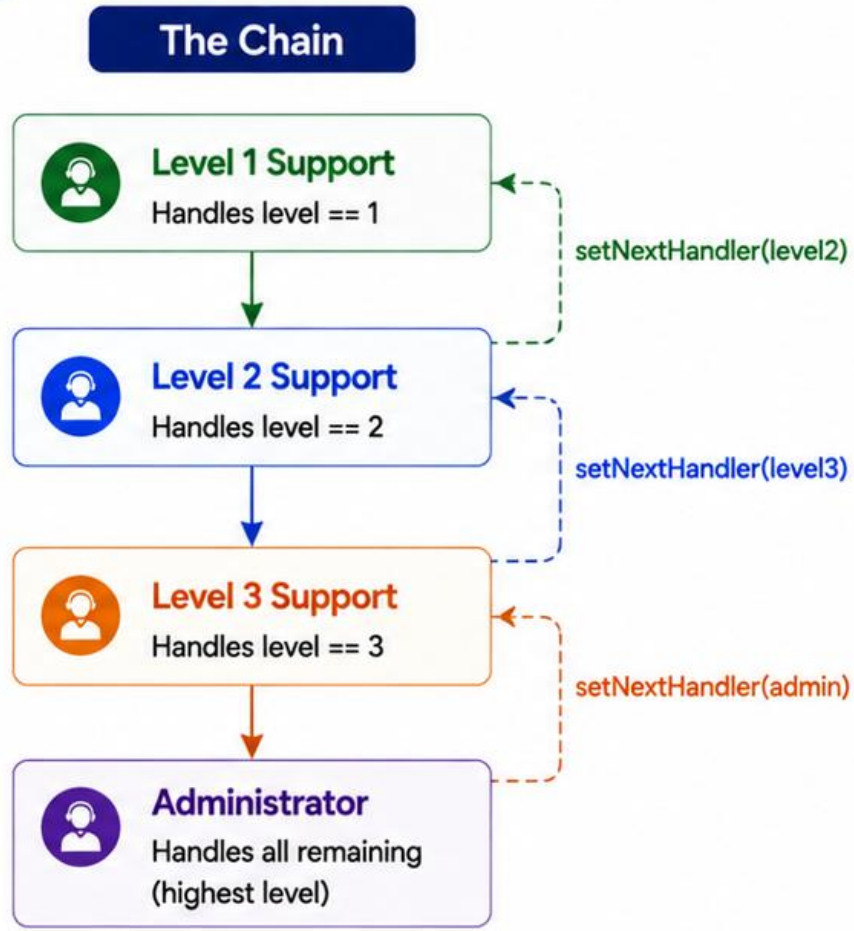
Java Code

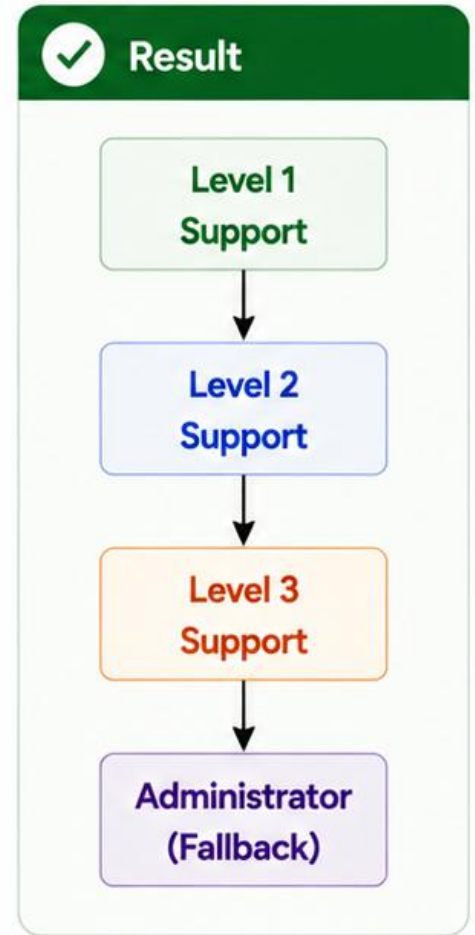
```java
Level1Support level1 = new Level1Support();
Level2Support level2 = new Level2Support();
Level3Support level3 = new Level3Support();
Administrator admin = new Administrator();

level1.setNextHandler(level2);
level2.setNextHandler(level3);
level3.setNextHandler(admin);

// Chain is ready!

// level1 is the entry point for all requests.
```





Key Takeaway

The client only knows the first handler (level1).
Each handler decides whether to handle the request or pass it to the next.

Sending a Request

The request travels along the chain **until a handler can process it.**

Java Code (Client)

```
SupportRequest request =
    new SupportRequest(
        3,
        "Network is unavailable"
    );

level1.handleRequest(request);
```

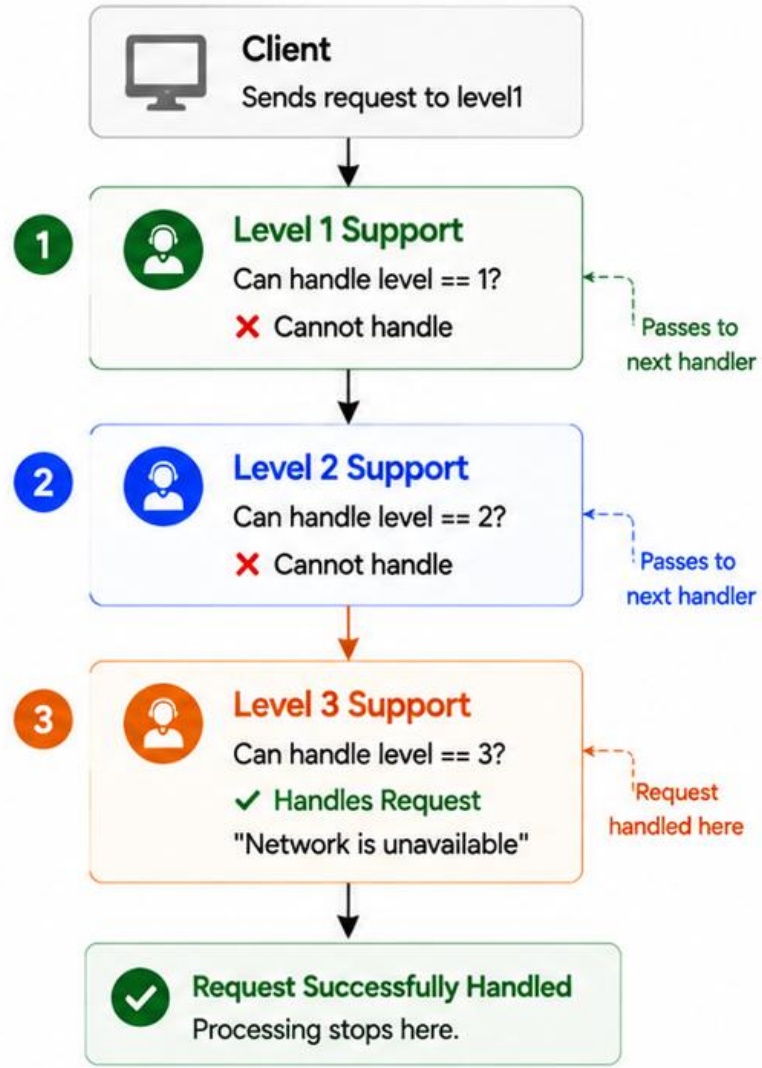
Request Details

- level: 3
- description: "Network is unavailable"

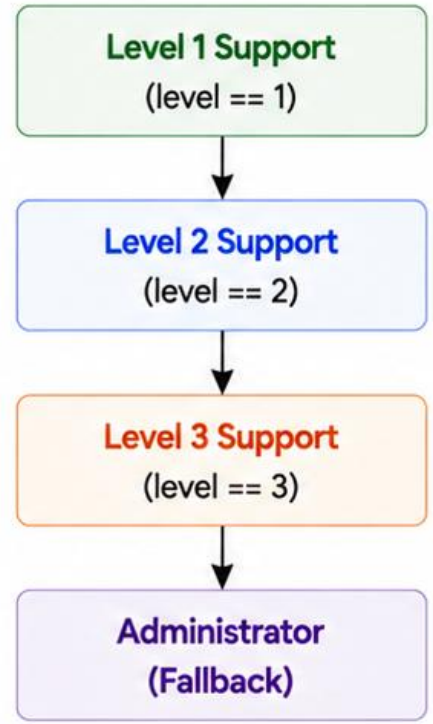
Key Takeaway

The client only knows where to send the request (level1).
It **does not know** which handler will process it.

Trace the Execution



The Handler Chain



What happened?

- Level 1 cannot handle (level is 3).
- Level 2 cannot handle (level is 3).
- Level 3 can handle (level is 3) → Processes it.

Reflection: Trace the Code

Before running the program, **trace how the request flows** through the chain of responsibility.

Code to Trace

```
SupportRequest request =
    new SupportRequest(
      2,
      "Software installation failed"
 );

level1.handleRequest(request);
```

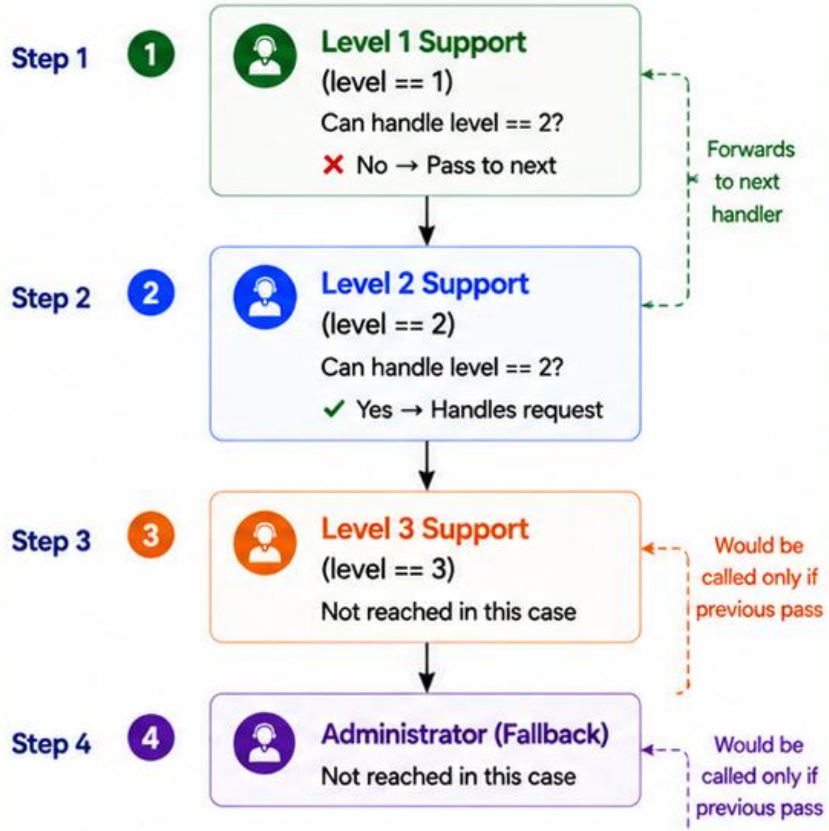
Assumptions

- Chain is built as:



- Each handler can handle only its matching level.
- If a handler cannot handle, it forwards the request to the next handler.

Trace the Execution (Predict Before Running)



Discuss

1. Which object receives the request first?

   _____

2. Which object handles the request?

   _____

3. Which objects are involved in processing?

   _____

4. What happens if no handler accepts the request?

   _____

5. What would happen if the chain order is changed?

   _____

Key Idea: The client only knows the entry point (level1). Each handler decides whether to handle or pass the request along the chain.

Problem 2: Expense Approval

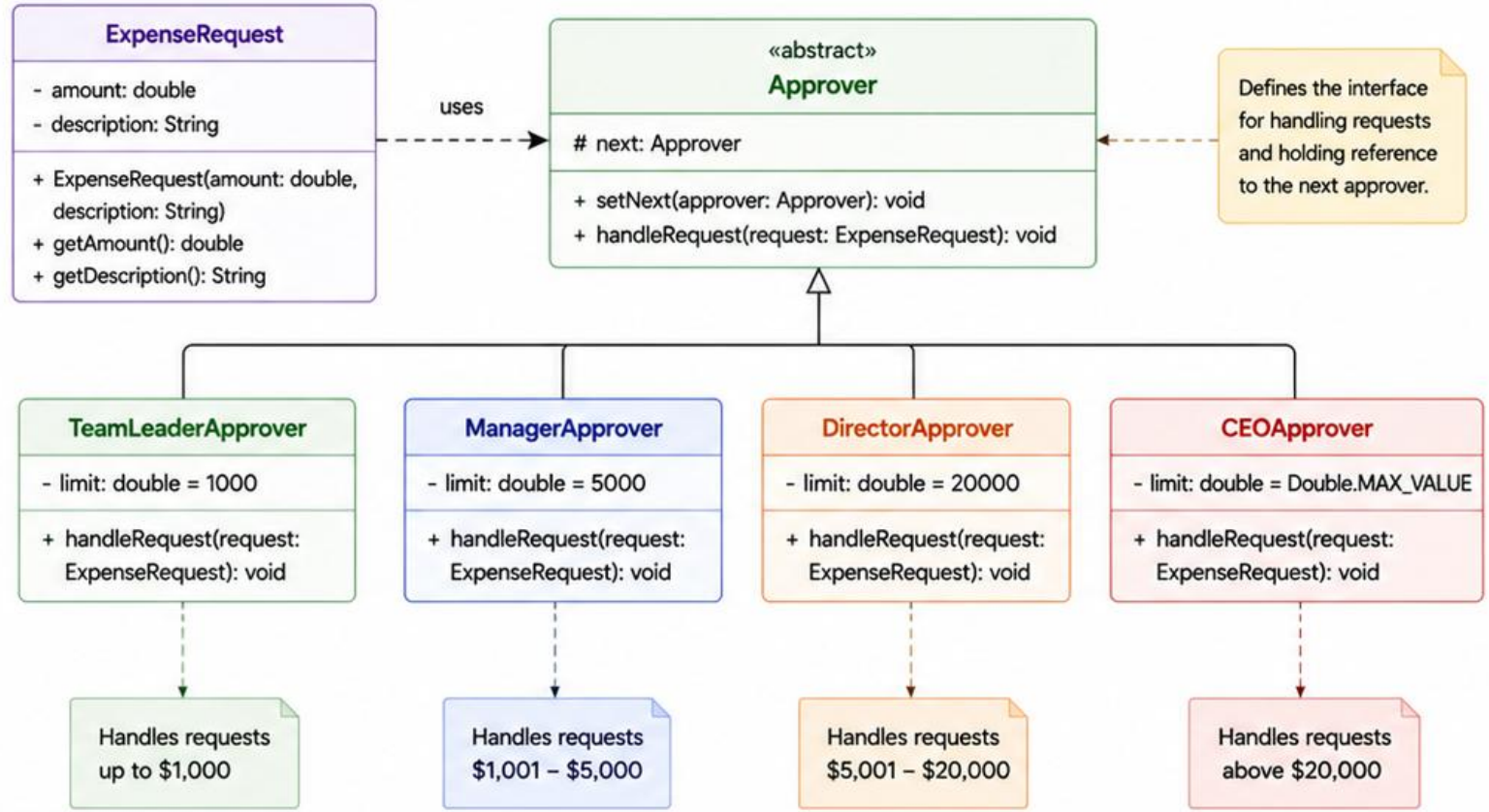
In-Class Group Activity

A company uses a hierarchical approval process for employee expense requests. Each approval level can approve requests within its limit. If a request exceeds the limit, it is forwarded to the next approver in the chain.

Approval Levels

Expense Amount	Approver
Up to $1,000	Team Leader
$1,001 – $5,000	Manager
$5,001 – $20,000	Director
Above $20,000	CEO

UML Class Diagram (Chain of Responsibility)



Approval Chain Flow



If a handler cannot approve the request, it forwards it to the next approver.

Problem 2: Expense Approval (Chain of Responsibility) – Java Implementation

1 ExpenseRequest.java

```java
public class ExpenseRequest {
    private double amount;
    private String description;

    public ExpenseRequest(double amount, String description) {
        this.amount = amount;
        this.description = description;
    }

    public double getAmount() {
        return amount;
    }

    public String getDescription() {
        return description;
    }
}
```

2 ExpenseApprover.java (Abstract Handler)

```java
public abstract class ExpenseApprover {
    protected ExpenseApprover nextApprover;

    public void setNextApprover(ExpenseApprover nextApprover) {
        this.nextApprover = nextApprover;
    }

    public final void approveRequest(ExpenseRequest request) {
        if (canApprove(request)) {
            processRequest(request);
        } else if (nextApprover != null) {
            nextApprover.approveRequest(request);
        } else {
            System.out.println("Request for $" + request.getAmount()
              + " cannot be approved. No appropriate approver available.");
        }
    }

    protected abstract boolean canApprove(ExpenseRequest request);
    protected abstract void processRequest(ExpenseRequest request);
}
```

3 TeamLeaderApprover.java

```java
public class TeamLeaderApprover extends ExpenseApprover {
    private static final double LIMIT = 5000.0;

    @Override
    protected boolean canApprove(ExpenseRequest request) {
        return request.getAmount() <= LIMIT;
    }

    @Override
    protected void processRequest(ExpenseRequest request) {
        System.out.println("Team Leader approved: $"
            + request.getAmount() + " - "
            + request.getDescription());
    }
}
```

4 ManagerApprover.java

```java
public class ManagerApprover extends ExpenseApprover {
    private static final double LIMIT = 5000.0;

    @Override
    protected boolean canApprove(ExpenseRequest request) {
        return request.getAmount() <= LIMIT;
    }

    @Override
    protected void processRequest(ExpenseRequest request) {
        System.out.println("Manager approved: $"
            + request.getAmount() + " - "
            + request.getDescription());
    }
}
```

5 DirectorApprover.java

```java
public class DirectorApprover extends ExpenseApprover {
    private static final double LIMIT = 20000.0;

    @Override
    protected boolean canApprove(ExpenseRequest request) {
        return request.getAmount() <= LIMIT;
    }

    @Override
    protected void processRequest(ExpenseRequest request) {
        System.out.println("Director approved: $"
            + request.getAmount() + " - "
            + request.getDescription());
    }
}
```

6 CEOApprover.java

```java
public class CEOApprover extends ExpenseApprover {

    @Override
    protected boolean canApprove(ExpenseRequest request) {
        return true; // CEO can approve any amount
    }

    @Override
    protected void processRequest(ExpenseRequest request) {
        System.out.println("CEO approved: $"
            + request.getAmount() + " - "
            + request.getDescription());
    }
}
```

7 Main.java (Example Usage)

```java
public class Main {
    public static void main(String[] args) {

        // Build the chain
        ExpenseApprover teamLeader = new TeamLeaderApprover();
        ExpenseApprover manager    = new ManagerApprover();
        ExpenseApprover director   = new DirectorApprover();
        ExpenseApprover ceo        = new CEOApprover();

        // Set up the chain: Team Leader -> Manager -> Director -> CEO
        teamLeader.setNextApprover(manager);
        manager.setNextApprover(director);
        director.setNextApprover(ceo);

        // Test requests
        ExpenseRequest[] requests = {
            new ExpenseRequest(750,   "Team Outing"),
            new ExpenseRequest(3500,  "Client Meeting"),
            new ExpenseRequest(12000, "System Upgrade"),
            new ExpenseRequest(50000, "New Office Setup")
        };

        for (ExpenseRequest request : requests) {
            System.out.println("\nRequest: $" + request.getAmount()
                + " - " + request.getDescription());
            teamLeader.approveRequest(request); // Entry point
        }
    }
}
```

Sample Output

```
Request: $750.0 - Team Outing
Team Leader approved: $750.0 - Team Outing

Request: $3500.0 - Client Meeting
Manager approved: $3500.0 - Client Meeting

Request: $12000.0 - System Upgrade
Director approved: $12000.0 - System Upgrade

Request: $50000.0 - New Office Setup
CEO approved: $50000.0 - New Office Setup
```

Approval Chain Flow



Key Takeaway

The Chain of Responsibility decouples the sender of a request from its receivers by giving multiple objects a chance to handle the request.

Main.java Explained – Chain of Responsibility (Expense Approval)

This class builds the chain of approvers and sends requests into the chain.

1 Create the Approvers

```
ExpenseApprover teamLeader = new TeamLeaderApprover();
ExpenseApprover manager    = new ManagerApprover();
ExpenseApprover director   = new DirectorApprover();
ExpenseApprover ceo        = new CEOApprover();
```

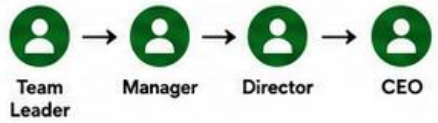
We create four approver objects.
All are referenced by the abstract type **ExpenseApprover**.
This demonstrates **polymorphism**.

2 Build the Chain

```
teamLeader.setNextApprover(manager);
manager.setNextApprover(director);
director.setNextApprover(ceo);
```

We link the approvers in order.
If one cannot approve, it passes the request to the next approver.

The Chain



3 Create Test Requests

```
ExpenseRequest[] requests = {
    new ExpenseRequest(750, "Team Outing"),
    new ExpenseRequest(3500, "Client Meeting"),
    new ExpenseRequest(12000, "System Upgrade"),
    new ExpenseRequest(50000, "New Office Setup")
};
```

An array of expense requests is created.
Each request has:

Amount + Description

Requests

#	Amount	Description
1	$750	Team Outing
2	$3,500	Client Meeting
3	$12,000	System Upgrade
4	$50,000	New Office Setup

4 Process Each Request

```
for (ExpenseRequest request : requests) {
   System.out.println("\nRequest: $" +
      request.getAmount() + " - " +
      request.getDescription());
  teamLeader.approveRequest(request);
}
```

For each request:

- Print the request details
- Send it to the first approver (teamLeader).

The client **only knows where the chain starts** – not who will finally approve.

5 Send the Request to the Chain

```
teamLeader.approveRequest(request);
```

The request enters the chain.
Each approver decides:

- Can I approve? If **YES** → Approve and **STOP**
- If **NO** → Pass to next approver

This is the heart of the Chain of Responsibility pattern!

How Requests Flow in the Chain (Examples)

1 $750 – Team Outing



Approved by **Team Leader**

2 $3,500 – Client Meeting



Approved by **Manager**

3 $12,000 – System Upgrade



Approved by **Director**

4 $50,000 – New Office Setup



Approved by **CEO**

Overall Flow of Main.java



Key Learning Point

Main.java only knows where to start the chain.
It does not need to decide who will ultimately approve the expense.

This is the power of the Chain of Responsibility – the sender is decoupled from the final receiver.

Priorities and Handler Selection

The effectiveness of the Chain of Responsibility depends on the **order of handlers.**
We must decide which handler should be tried first.

Possible Criteria for Ordering

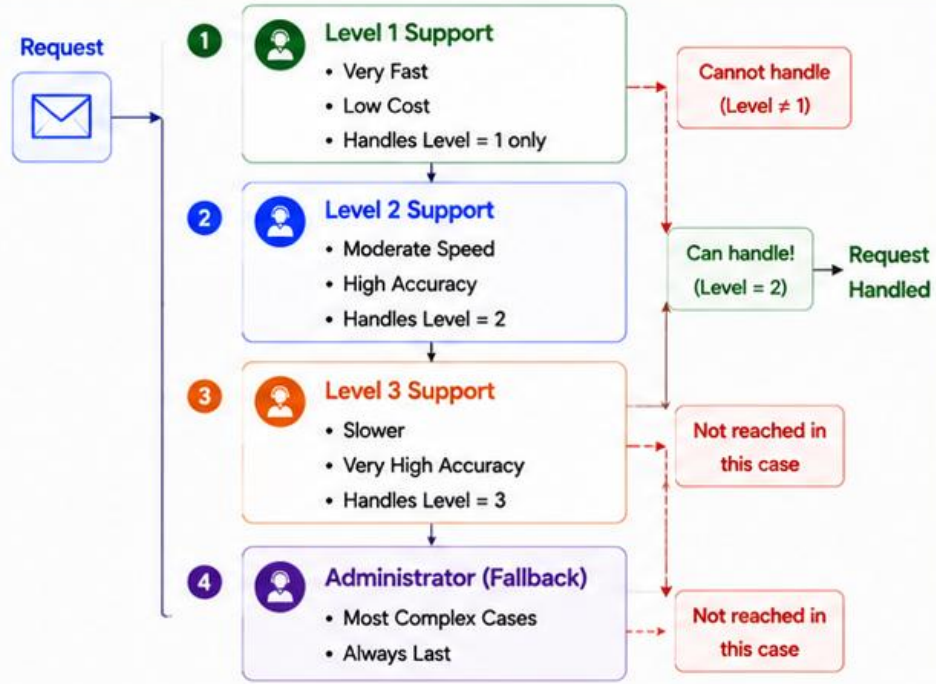
- **Performance** — Handles requests quickly
- **Cost** — Lower operational cost
- **Accuracy** — Produces the most correct results
- **Availability** — More likely to be available
- **Complexity** — Can handle more complex cases
- **Severity** — Handle high-severity first
- **Priority** — Business or domain priority

Key Takeaway

There is **no one-size-fits-all** order.
Choose an order that best balances the criteria and the goals of your system.

Example: Support Request Chain

Example request: Level = 2 (Software installation failed)



In this example, **Level 2 Support** handles the request.
Handlers before it were checked; those after it were not needed.

Reflection

Should the **fastest** handler always be placed **first**?

What if the **most accurate** handler is slower?

Think About...

- What is more important for this system: speed, accuracy, or cost?
- Are low-level requests more frequent than high-level ones?
- Does the severity of the issue change your ordering?
- Is a fallback handler always required at the end?

Remember: Ordering affects performance, correctness, cost, and overall user satisfaction.
Review and adjust the chain order as system needs evolve.

Exception Handling Analogy

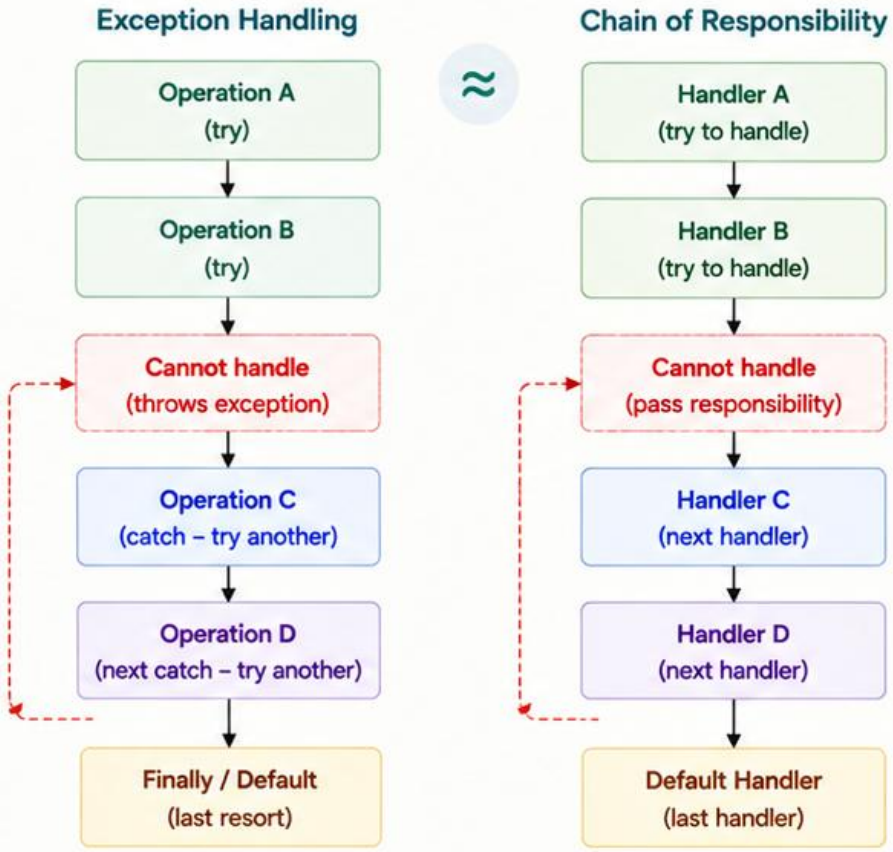
Exception handling in code is very similar to the Chain of Responsibility pattern.
When one block cannot handle the situation, control is passed to the next alternative.

Exception Handling (Try–Catch Analogy)

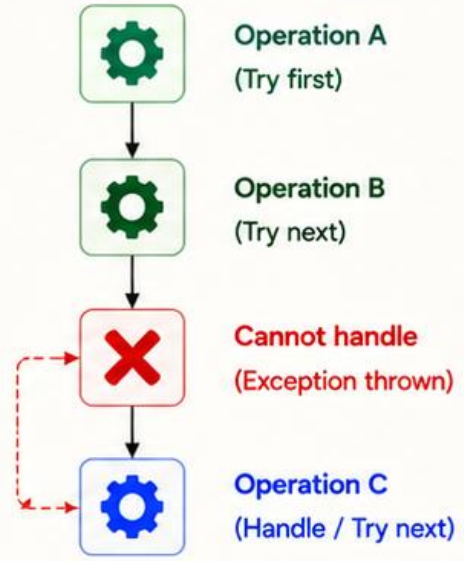
```
1  try {
2     operationB();
3  }
4  catch (Exception e) {
5     // Try another handler
6     operationC();
7  }
8  catch (AnotherException ex) {
9     // Try another handler
10    operationD();
```

- We TRY operationB() first.
- If it cannot handle (throws an exception), control moves to the catch block.
- In the catch block, we TRY another alternative (operationC()).
- If that also fails, we move to the next catch, and so on...

Conceptual Comparison



Visual Analogy



Discussion

Can exception handling be viewed as a form of passing responsibility?

What are the similarities?
What are the differences?

Key Takeaway: Both approaches provide alternative ways to handle a request when the current option cannot.
They improve robustness, flexibility, and maintainability.

Applications: Where Can We Use Chain of Responsibility?

The request is passed along a chain of handlers until one of them handles it. This **decouples sender and receiver and provides flexibility.**

Common Application Areas

1 Email Spam Detectors

Emails pass through multiple filters (blacklist, rules, ML model, content analysis) until one marks it as spam.

2 Medical Testing

A series of tests are performed one after another. If one test confirms the condition, the process stops.

3 Troubleshooting

Support issues move from Level 1 to Level 2 to Level 3 and finally to an expert if not resolved earlier.

4 Classifier Systems

An input is evaluated by a sequence of classifiers until one classifies it with sufficient confidence.

5 Event Handling Frameworks

Events are passed through a chain of handlers/listeners until one handles the event.

6 Security Systems

Requests are checked by multiple security layers (authentication, authorization, threat detection, logging, etc.).

Additional Real-World Examples

IT Support Escalation

Requests are handled by support agents in increasing levels until the issue is resolved.

Expense Approval

Expense requests move from Team Leader → Manager → Director → CEO based on amount.

Authentication & Authorization Pipelines

A request passes through authentication, role check, permissions check, policy check, etc., until it is either allowed or denied.

Fraud Detection

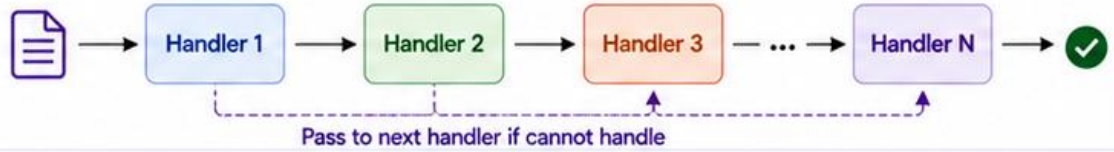
Transactions are analyzed by multiple rules, models, and services; if one flags it as fraud, processing stops.

Request Filtering

Incoming requests go through filters (rate limit, IP blocking, content filtering, validation) until accepted or rejected.

Key Takeaway

Use Chain of Responsibility when you want to decouple the sender from the receiver and give multiple objects a chance to handle a request.



Advantages of Chain of Responsibility

The Chain of Responsibility pattern provides a **flexible** way to **pass a request** along a chain of handlers until **one of them handles it.**

1	**Multiple Handlers**	More than one object can potentially handle a request.
2	**Flexibility**	Handlers can be added or removed easily.
3	**Dynamic Ordering**	The order of handlers may be changed.
4	**Separation of Responsibility**	Each handler focuses only on the type of request it can handle.
5	**Loose Coupling**	The sender does not need to know the final receiver.

Example: In an IT support system, Level 1, Level 2, and Level 3 agents can all potentially resolve the issue.

Example: New approval levels or validation rules can be added without changing existing code.

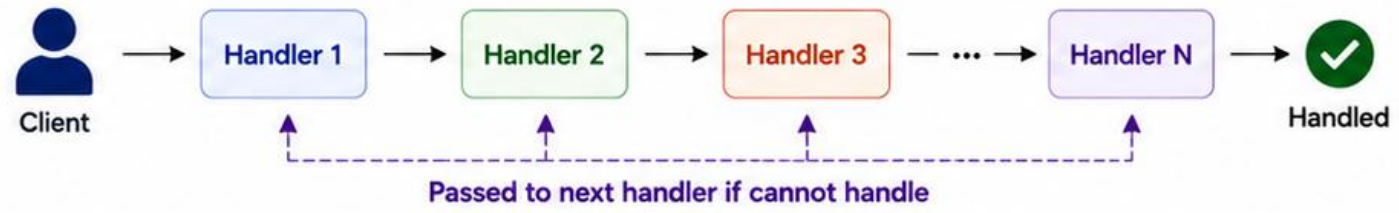
Example: High-priority checks can be placed earlier in the chain based on business needs.

Example: Each handler contains the logic for a specific task, making the system easier to maintain.

Example: The client sends the request to the first handler and the rest is managed by the chain.

Key Takeaway

Chain of Responsibility promotes **flexibility, extensibility,** and **maintainability** by decoupling the sender from receivers and allowing multiple objects a chance to handle a request.



Challenges of Chain of Responsibility

While Chain of Responsibility provides **flexibility**, it also introduces some **challenges** that must be carefully managed.

1 No Guarantee of Handling

A request may reach the end of the chain without being handled by any object.

Example: If no handler can process the request (e.g., unknown request type), it may be dropped or requires a default handler.

2 Debugging Can Be Difficult

A request may pass through several handlers before being handled (or not handled), making it harder to trace the flow.

Example: It may be unclear which handler modified the request or why the final handler was chosen.

3 Inefficiency

Unnecessary handlers may process the request even though they cannot handle it, increasing processing time.

Example: If the chain is long and most handlers just pass the request along, performance can be negatively impacted.

4 Order Matters

Changing priorities or the order of handlers may affect the outcome.

Example: Placing a general handler before a specific one may prevent the specific handler from ever seeing the request.

5 Maintenance Overhead

As the chain grows, managing and maintaining handlers and their order becomes complex.

Example: More handlers mean more code to maintain, configure, and test.

Reflection

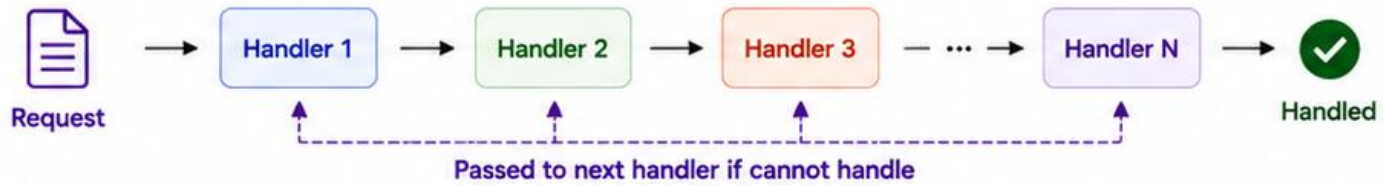
Is a Chain of Responsibility always better than a simple if-else statement?

- Use Chain of Responsibility when you need flexibility, extensibility, and multiple potential handlers.
- Use if-else when the logic is simple, fixed, and unlikely to change.

The right choice depends on the context, complexity, and future growth.

Key Takeaway

Use Chain of Responsibility when you need a **flexible** and **extensible** way to handle requests, but be aware of **performance, ordering,** and **maintenance** trade-offs.



Beyond OOP: From Handlers to AI Agents

The Chain of Responsibility principle is not limited to objects.
It can be extended to **AI agents**, where specialised agents **collaborate** to handle complex requests.

The Idea

A user request flows through a chain of specialised AI agents.
Each agent decides whether it can handle the request.
If not, it forwards the request to the next suitable agent.

Why This Matters

- Modular & specialised capabilities
- Scalable and extensible
- Easy to add or remove agents
- Clear separation of concerns
- Robust handling of complex tasks

Key Takeaway

From methods to objects,
from objects to agents — the principle remains:
Pass the request until someone can handle it.



Each Agent Asks

"Can I handle this request?"

If YES

Process the request and return the result.

If NO

Forward the request to another suitable agent in the chain.

Example Scenario

User: "Give me a monthly sales performance report"

- **Classification Agent:** Identifies as "Report Request" → No
- **Data Agent:** Collects sales data → No
- **Analysis Agent:** Analyzes performance → No
- **Reporting Agent:** Generates the report → **Yes (Handled)**

Chain of Responsibility vs AI Agents

Both approaches aim to **route a request** to the most suitable handler, but they differ in how the routing, reasoning, and collaboration are achieved.

Chain of Responsibility		Agent-Based System
Usually predefined chain The sequence of handlers is defined in advance.	vs	**Can dynamically select agents** Agents may be discovered, selected, or activated at runtime based on context.
Sequential forwarding A request moves from one handler to the next in a fixed order.	vs	**May communicate in multiple ways** Agents can communicate in parallel, broadcast, request, or publish/subscribe.
Fixed responsibilities Each handler has a specific, static responsibility.	vs	**May reason about responsibilities** Agents can understand tasks, reason, and decide how best to handle them.
Handle or forward A handler either processes the request or forwards it.	vs	**Can collaborate and negotiate** Agents can work together, share tasks, negotiate, and reach agreements.
Usually predictable flow The path of a request is deterministic and easy to trace.	vs	**May dynamically change the flow** The workflow can adapt based on goals, feedback, learning, or environment.

How They Relate

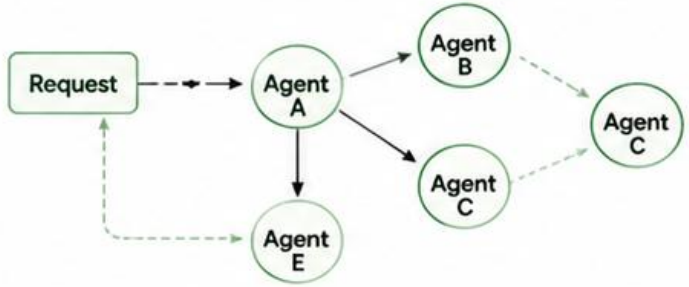
Chain of Responsibility (CoR)

Linear forwarding until handled



Agent-Based System

Flexible interactions among agents



Important Discussion

An Agentic AI system may use Chain of Responsibility as one of the design patterns, but it is **not limited** to this pattern.

Key Takeaway

Chain of Responsibility is a simple, reliable pattern for ordered handling.

Agentic AI systems go beyond it by enabling reasoning, collaboration, and adaptive workflows.

CoR → AI Agents → Reasoning → Collaboration → Adaptation

Final Reflection

Think – Discuss – Share

Take a few minutes to think about the following questions.
Discuss with your peers and be ready to share your thoughts with the class.

Think | Discuss | Share

1. How does the Chain of Responsibility reduce coupling between the sender and receiver?

2. When would a simple if-else approach be better than Chain of Responsibility?

3. What problems can occur if the order of handlers is changed?

4. How could you ensure that a request does not disappear when it reaches the end of the chain?

5. Can multiple handlers process the same request? How would you modify the design?

6. How does an AI Agent system extend the idea of a traditional Chain of Responsibility?

Be prepared to share key points from your discussion!

There are many valid perspectives – listen, learn, and build on each other's ideas.